Proyecto 1
Biblioteca de generación y manejo de grafos

Josue gutierrez Andrade

Diseño y Análisis de Algoritmos

Resumen

Se presenta una biblioteca orientada a objetos, escrita en Python 3, para describir y generar grafos. La biblioteca define las clases `Nodo`, `Arista` y `Grafo`, e implementa seis modelos de generación: malla, Erdős–Rényi, Gilbert, geográfico simple, Barabási–Albert y Dorogovtsev–Mendes. Para cada modelo se generaron grafos de 50, 200 y 500 nodos, que se exportaron a formato GraphViz y se visualizaron con Gephi.

Índice

1. Estructura de la biblioteca **2**

2. Representación de los grafos **2**
- 2.1. Clase `Nodo` . . . 2
- 2.2. Clase `Arista` . . . 2
- 2.3. Clase `Grafo` . . . 2
- 2.4. Métodos de `Grafo` . . . 3

3. Generadores de grafos **3**
- 3.1. Malla: `grafo_malla(m, n)` . . . 4
- 3.2. Erdős–Rényi: `grafo_erdos_renyi(n, m)` . . . 4
- 3.3. Gilbert: `grafo_gilbert(n, p)` . . . 5
- 3.4. Geográfico simple: `grafo_geografico(n, r)` . . . 6
- 3.5. Barabási–Albert: `grafo_barabasi_albert(n, d)` . . . 6
- 3.6. Dorogovtsev–Mendes: `grafo_dorogovtsev_mendes(n)` . . . 7
- 3.7. Resumen . . . 8

4. Ejemplo de uso **8**

5. Generación de los archivos GraphViz **8**

6. Visualización de los grafos **9**

1. Estructura de la biblioteca

El código está organizado en el paquete `grafos` y un script de apoyo:

Archivo	Contenido
`grafos/nodo.py`	Clase `Nodo`
`grafos/arista.py`	Clase `Arista`
`grafos/grafo.py`	Clase `Grafo`
`grafos/generadores.py`	Los seis generadores de grafos
`generar_gv.py`	Script que produce los 18 archivos `.gv`
`gv/`	Archivos GraphViz generados
`imagenes/`	Visualizaciones hechas con Gephi

Tabla 1: Archivos del proyecto.

2. Representación de los grafos

2.1. Clase `Nodo`

Un nodo solo almacena su identificador, `id_nodo`. La igualdad y el *hash* se definen a partir de ese identificador, de modo que dos objetos `Nodo` con el mismo identificador se consideran el mismo nodo. El identificador puede ser cualquier valor *hashable*: los modelos aleatorios usan enteros $0, 1, \ldots, n-1$ y la malla usa tuplas (i, j).

2.2. Clase `Arista`

Una arista guarda referencias a sus nodos `origen` y `destino`, una bandera `dirigida` y un `peso` opcional (por omisión `None`).

2.3. Clase `Grafo`

El grafo se representa con una **lista de adyacencia basada en conjuntos**, acompañada de una lista de aristas. La clase mantiene cuatro estructuras:

Atributo	Tipo	Contenido
`nodos`	`dict`	identificador → objeto `Nodo`
`aristas`	`list`	objetos `Arista`, en orden de inserción
`_vecinos`	`dict` de `set`	identificador → identificadores adyacentes
`_grado`	`dict`	identificador → número de aristas incidentes

Tabla 2: Atributos de la clase `Grafo`.

Usar conjuntos en `_vecinos` permite consultar si una arista existe en tiempo $O(1)$ promedio, lo que resulta esencial para los generadores, que hacen esa consulta cada vez que intentan agregar una arista. La lista `aristas` es la única estructura que conserva el peso y la dirección de cada arista, y es la que se recorre al exportar el grafo.

2.4. Métodos de `Grafo`

`agregar_nodo(id_nodo)` Crea el nodo si no existe, inicializa su conjunto de vecinos vacío y su grado en cero, y devuelve el objeto `Nodo`. Si el nodo ya existía, devuelve el existente sin modificar nada.

`existe_arista(id_nodo1, id_nodo2)` Devuelve `True` si `id_nodo2` está en el conjunto de vecinos de `id_nodo1`.

`agregar_arista(id_nodo1, id_nodo2, peso=None)` Agrega la arista y la devuelve. Si la arista es un lazo o ya existe, no hace nada y devuelve `None`; por lo tanto, los grafos de la biblioteca son siempre *simples*. Los generadores aprovechan este valor de retorno para saber si la arista realmente se creó.

`vecinos(id_nodo)` Devuelve el conjunto de identificadores adyacentes.

`grado(id_nodo)` Devuelve el número de aristas incidentes en el nodo. En grafos dirigidos cuenta tanto las aristas de entrada como las de salida.

`es_conexo()` Hace un recorrido en profundidad iterativo (con una pila) desde un nodo cualquiera y verifica que se alcancen todos los nodos. Ignora la dirección de las aristas, así que en grafos dirigidos evalúa conexidad débil. Su costo es $O(|V| + |E|)$.

`guardar_graphviz(archivo)` Escribe el grafo en formato GraphViz: primero todos los nodos y después todas las aristas, usando `graph` y `-` para grafos no dirigidos, o `digraph` y `->` para dirigidos.

```python
def agregar_arista(self, id_nodo1, id_nodo2, peso=None):
    self.agregar_nodo(id_nodo1)
    self.agregar_nodo(id_nodo2)
    if id_nodo1 == id_nodo2 or self.existe_arista(id_nodo1, id_nodo2):
        return None
    arista = Arista(self.nodos[id_nodo1], self.nodos[id_nodo2], self.dirigido, peso)
    self.aristas.append(arista)
    self._vecinos[id_nodo1].add(id_nodo2)
    if not self.dirigido:
        self._vecinos[id_nodo2].add(id_nodo1)
    self._grado[id_nodo1] += 1
    self._grado[id_nodo2] += 1
    return arista
```

Código 1: Método `agregar_arista` de la clase `Grafo`.

En un grafo no dirigido la adyacencia se registra en ambos sentidos, pero la arista aparece una sola vez en la lista `aristas`. En un grafo dirigido, `_vecinos[u]` contiene únicamente los sucesores de u, por lo que las aristas $u \to v$ y $v \to u$ pueden coexistir.

3. Generadores de grafos

Todos los generadores reciben un parámetro opcional `dirigido` (por omisión `False`) y devuelven un objeto `Grafo`. Cinco de los seis modelos son aleatorios y usan el módulo estándar `random`; como no se fija una semilla, cada ejecución produce un grafo distinto.

3.1. Malla: `grafo_malla(m, n)`

Genera una malla de $m \times n$ nodos. Cada nodo se identifica con la tupla (i, j), con $0 \leq i < m$ y $0 \leq j < n$, y se conecta con el nodo $(i+1, j)$ y con el nodo $(i, j+1)$ cuando estos existen. Es el único modelo determinista.

```
def grafo_malla(m, n, dirigido=False):
    grafo = Grafo(dirigido)
    for i in range(m):
        for j in range(n):
            grafo.agregar_nodo((i, j))
    for i in range(m):
        for j in range(n):
            if i < m - 1:
                grafo.agregar_arista((i, j), (i + 1, j))
            if j < n - 1:
                grafo.agregar_arista((i, j), (i, j + 1))
    return grafo
```

Código 2: Generador de malla.

El grafo resultante tiene mn nodos y $n(m-1)+m(n-1) = 2mn-m-n$ aristas, y se construye en tiempo $\Theta(mn)$.

Ejemplo. La llamada `grafo_malla(2, 3)` produce una malla de 6 nodos y $2 \cdot 6 - 2 - 3 = 7$ aristas. Al guardarla con `guardar_graphviz` se obtiene el siguiente archivo:

```
graph G {
    "(0, 0)";
    "(0, 1)";
    "(0, 2)";
    "(1, 0)";
    "(1, 1)";
    "(1, 2)";
    "(0, 0)" -- "(1, 0)";
    "(0, 0)" -- "(0, 1)";
    "(0, 1)" -- "(1, 1)";
    "(0, 1)" -- "(0, 2)";
    "(0, 2)" -- "(1, 2)";
    "(1, 0)" -- "(1, 1)";
    "(1, 1)" -- "(1, 2)";
}
```

Código 3: Archivo `.gv` de la malla de 2×3.

3.2. Erdős–Rényi: `grafo_erdos_renyi(n, m)`

Crea n nodos y elige m aristas distintas uniformemente al azar. En cada iteración se toma un par de nodos distintos con `random.sample` y se intenta agregar la arista; solo se cuenta si `agregar_arista` la creó, es decir, si no estaba repetida.

```
def grafo_erdos_renyi(n, m, dirigido=False):
    max_m = n * (n - 1) if dirigido else n * (n - 1) // 2
    if m > max_m:
```

```
        raise ValueError(f"m={m} excede el máximo de aristas posibles ({max_m}) para n={n
    }")

    grafo = Grafo(dirigido)
    for i in range(n):
        grafo.agregar_nodo(i)

    aristas_creadas = 0
    while aristas_creadas < m:
        u, v = random.sample(range(n), 2)
        if grafo.agregar_arista(u, v) is not None:
            aristas_creadas += 1
    return grafo
```

Código 4: Generador de Erdős–Rényi.

La validación inicial evita un ciclo infinito: si se pidieran más aristas que las $M = \binom{n}{2}$ posibles (o $M = n(n-1)$ en el caso dirigido), el ciclo nunca terminaría. Cuando ya hay k aristas, un intento tiene éxito con probabilidad $1 - k/M$, de modo que el número esperado de intentos es

$$\sum_{k=0}^{m-1} \frac{M}{M-k},$$

que es cercano a m cuando el grafo es disperso ($m \ll M$) y crece conforme m se aproxima a M.

Ejemplo. `grafo_erdos_renyi(50, 148)` genera un grafo de 50 nodos con exactamente 148 aristas.

3.3. Gilbert: `grafo_gilbert(n, p)`

Crea n nodos y, para cada par posible, agrega la arista con probabilidad p de forma independiente. En el caso no dirigido cada par $\{i, j\}$ se considera una sola vez (con $i < j$); en el dirigido se consideran los dos pares ordenados.

```
def grafo_gilbert(n, p, dirigido=False):
    grafo = Grafo(dirigido)
    for i in range(n):
        grafo.agregar_nodo(i)

    for i in range(n):
        for j in range(n):
            if i == j or (not dirigido and j < i):
                continue
            if random.random() < p:
                grafo.agregar_arista(i, j)
    return grafo
```

Código 5: Generador de Gilbert.

A diferencia del modelo de Erdős–Rényi, el número de aristas no es fijo: es una variable aleatoria binomial con valor esperado $p\binom{n}{2}$ en el caso no dirigido. El costo es $\Theta(n^2)$, porque se revisan todos los pares.

Ejemplo. `grafo_gilbert(50, 0.1)` genera un grafo de 50 nodos con alrededor de $0.1 \cdot \binom{50}{2} = 122.5$ aristas en promedio.

3.4. Geográfico simple: `grafo_geografico(n, r)`

Asigna a cada nodo una posición (x, y) uniforme en el cuadrado unitario y conecta dos nodos cuando su distancia euclidiana es menor o igual que r.

```
def grafo_geografico(n, r, dirigido=False):
    grafo = Grafo(dirigido)
    posiciones = [(random.random(), random.random()) for _ in range(n)]
    for i in range(n):
        grafo.agregar_nodo(i)

    for i in range(n):
        for j in range(n):
            if i == j or (not dirigido and j < i):
                continue
            dx = posiciones[i][0] - posiciones[j][0]
            dy = posiciones[i][1] - posiciones[j][1]
            if math.hypot(dx, dy) <= r:
                grafo.agregar_arista(i, j)
    return grafo
```

Código 6: Generador geográfico simple.

La aleatoriedad está solo en las posiciones; una vez fijadas, las aristas quedan determinadas. Las posiciones se usan únicamente durante la generación y no se guardan en el grafo. El costo es $\Theta(n^2)$, por la comparación de todos los pares.

Ejemplo. `grafo_geografico(50, 0.22)` genera un grafo de 50 nodos en el que se conectan los nodos situados a distancia 0.22 o menor.

3.5. Barabási–Albert: `grafo_barabasi_albert(n, d)`

Se implementa la variante del modelo en la que d es el grado máximo de cada nodo. Los primeros d nodos se conectan todos entre sí. Después, cada nodo nuevo recorre los nodos existentes en orden aleatorio y se conecta con cada candidato v con probabilidad

$$P(v) = 1 - \frac{\text{grado}(v)}{d},$$

deteniéndose cuando el nodo nuevo alcanza d aristas.

```
def grafo_barabasi_albert(n, d, dirigido=False):
    grafo = Grafo(dirigido)
    for i in range(min(d, n)):
        grafo.agregar_nodo(i)
    for i in range(min(d, n)):
        for j in range(i + 1, min(d, n)):
            grafo.agregar_arista(i, j)

    for nuevo in range(d, n):
        grafo.agregar_nodo(nuevo)
```

```
        candidatos = list(range(nuevo))
        random.shuffle(candidatos)
        for objetivo in candidatos:
            if grafo.grado(nuevo) >= d:
                break
            probabilidad = 1 - grafo.grado(objetivo) / d
            if random.random() < probabilidad:
                grafo.agregar_arista(nuevo, objetivo)
    return grafo
```

Código 7: Generador de Barabási–Albert.

Un nodo que ya tiene grado d tiene probabilidad cero de recibir más aristas, por lo que ningún nodo supera el grado d y el grafo tiene a lo más $nd/2$ aristas. El orden aleatorio de los candidatos evita favorecer siempre a los nodos con identificador más bajo. Cada candidato se prueba una sola vez, así que un nodo nuevo puede terminar con menos de d aristas. Como cada nodo nuevo recorre hasta todos los anteriores, el costo es $O(n^2)$.

Ejemplo. `grafo_barabasi_albert(50, 4)` genera un grafo de 50 nodos con grado máximo 4 y, por lo tanto, a lo más 100 aristas.

3.6. Dorogovtsev–Mendes: `grafo_dorogovtsev_mendes(n)`

Comienza con un triángulo formado por los nodos 0, 1 y 2. Cada nodo nuevo elige una arista existente uniformemente al azar y se conecta con sus dos extremos, con lo que forma un nuevo triángulo.

```
def grafo_dorogovtsev_mendes(n, dirigido=False):
    grafo = Grafo(dirigido)
    for i in range(3):
        grafo.agregar_nodo(i)
    grafo.agregar_arista(0, 1)
    grafo.agregar_arista(1, 2)
    grafo.agregar_arista(2, 0)

    for nuevo in range(3, n):
        grafo.agregar_nodo(nuevo)
        arista = random.choice(grafo.aristas)
        grafo.agregar_arista(nuevo, arista.origen.id_nodo)
        grafo.agregar_arista(nuevo, arista.destino.id_nodo)
    return grafo
```

Código 8: Generador de Dorogovtsev–Mendes.

Cada nodo nuevo aporta exactamente dos aristas, así que el grafo tiene siempre $3 + 2(n-3) = 2n-3$ aristas y es conexo por construcción. Elegir una arista al azar equivale a elegir sus extremos con probabilidad proporcional a su grado, lo que favorece a los nodos mejor conectados. Como `random.choice` toma tiempo constante, el costo total es $\Theta(n)$.

Ejemplo. `grafo_dorogovtsev_mendes(50)` genera un grafo de 50 nodos y $2 \cdot 50 - 3 = 97$ aristas.

3.7. Resumen

Modelo	Parámetros	Aristas	Costo
Malla	m, n	$2mn - m - n$	$\Theta(mn)$
Erdős–Rényi	n, m	m	$\approx m$ intentos si $m \ll \binom{n}{2}$
Gilbert	n, p	$p\binom{n}{2}$ en promedio	$\Theta(n^2)$
Geográfico simple	n, r	aleatorio	$\Theta(n^2)$
Barabási–Albert	n, d	a lo más $nd/2$	$O(n^2)$
Dorogovtsev–Mendes	n	$2n - 3$	$\Theta(n)$

Tabla 3: Resumen de los generadores (caso no dirigido).

4. Ejemplo de uso

El siguiente programa genera un grafo con el modelo de Gilbert, consulta algunas de sus propiedades y lo guarda en formato GraphViz:

```
from grafos import grafo_gilbert

g = grafo_gilbert(50, 0.1)

print(g)                # Grafo(nodos=50, aristas=..., dirigido=False)
print(g.grado(0))       # grado del nodo 0
print(g.vecinos(0))     # conjunto de vecinos del nodo 0
print(g.es_conexo())    # True si todos los nodos están conectados

g.guardar_graphviz("mi_grafo.gv")
```

Código 9: Uso básico de la biblioteca.

5. Generación de los archivos GraphViz

El script `generar_gv.py` genera 18 archivos `.gv`: tres tamaños (50, 200 y 500 nodos) por cada uno de los seis modelos. Para que las visualizaciones muestren una sola componente, cada grafo se vuelve a generar hasta que `es_conexo()` devuelve `True`, con un máximo de 1000 intentos.

Los parámetros de los modelos aleatorios se eligieron ligeramente por encima del umbral de conectividad, de modo que el grafo resulte conexo casi siempre:

Modelo	Parámetro usado
Erdős–Rényi	$m = \text{round}\left(\dfrac{n(\ln n + 2)}{2}\right)$
Gilbert	$p = \dfrac{\ln n + 3}{n}$
Geográfico simple	$r = \sqrt{\dfrac{\ln n + 4}{\pi n}}$
Barabási–Albert	$d = 4$
Malla	5×10, 10×20 y 20×25

Tabla 4: Parámetros usados en `generar_gv.py`.

La tabla 5 muestra el número de aristas de los grafos generados.

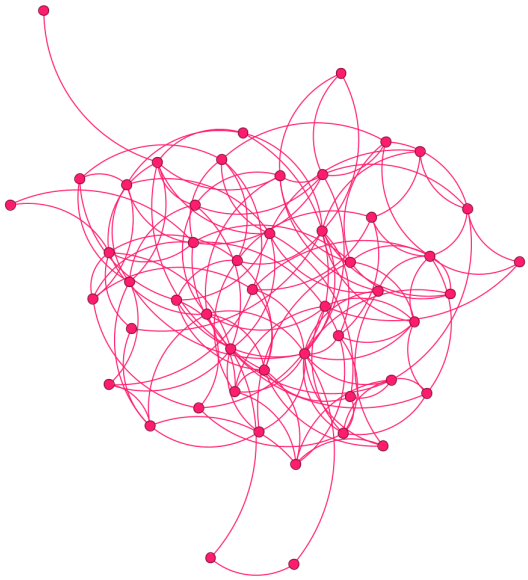
	Aristas		
Modelo	$n = 50$	$n = 200$	$n = 500$
Erdős–Rényi	148	730	2054
Gilbert	178	814	2241
Geográfico simple	179	817	2445
Barabási–Albert	96	395	996
Dorogovtsev–Mendes	97	397	997
Malla	85	370	955

Tabla 5: Número de aristas de los 18 grafos generados.

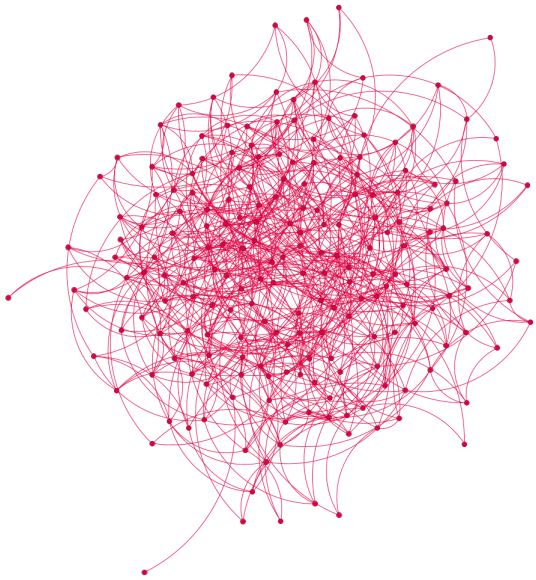
Los valores coinciden con lo esperado: Erdős–Rényi tiene exactamente el número de aristas solicitado, Dorogovtsev–Mendes tiene $2n - 3$, la malla tiene $2mn - m - n$ y Barabási–Albert queda apenas por debajo de su cota $nd/2$ (100, 400 y 1000).

6. Visualización de los grafos

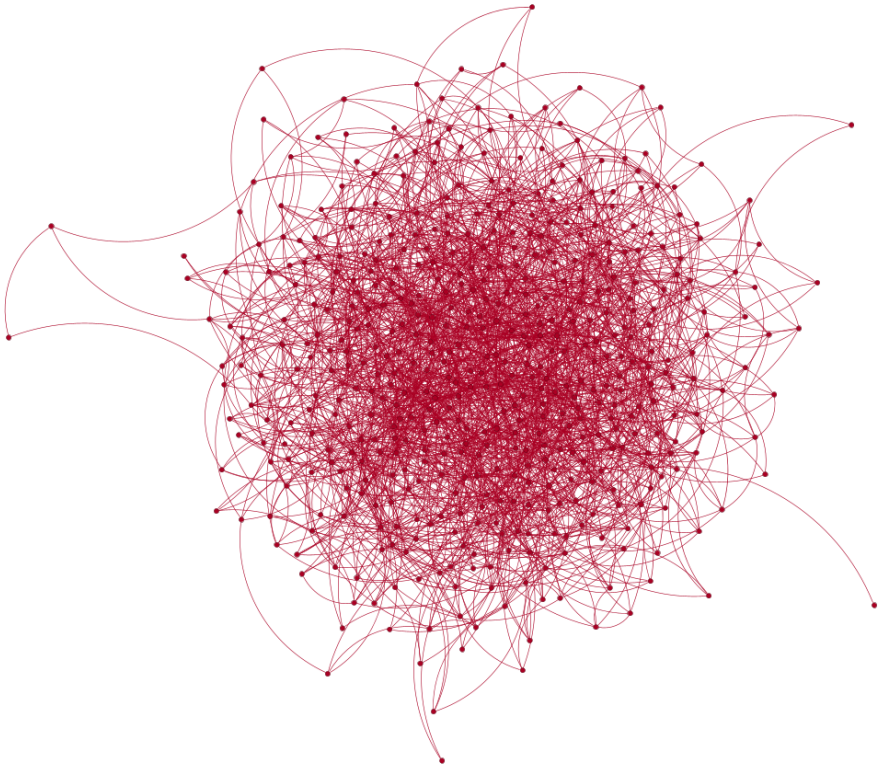
Los archivos `.gv` se visualizaron con Gephi. Las figuras 1 a 6 muestran los 18 grafos generados.



(a) $n = 50$

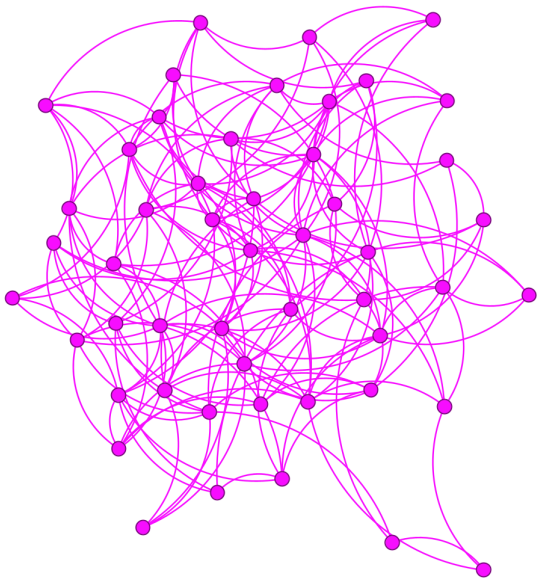


(b) $n = 200$

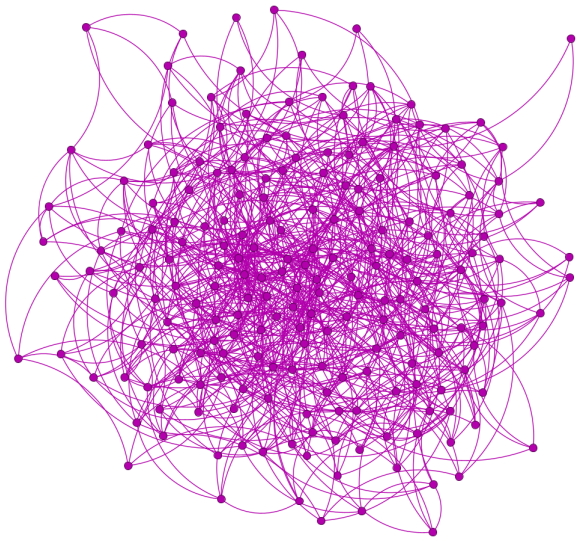


(c) $n = 500$

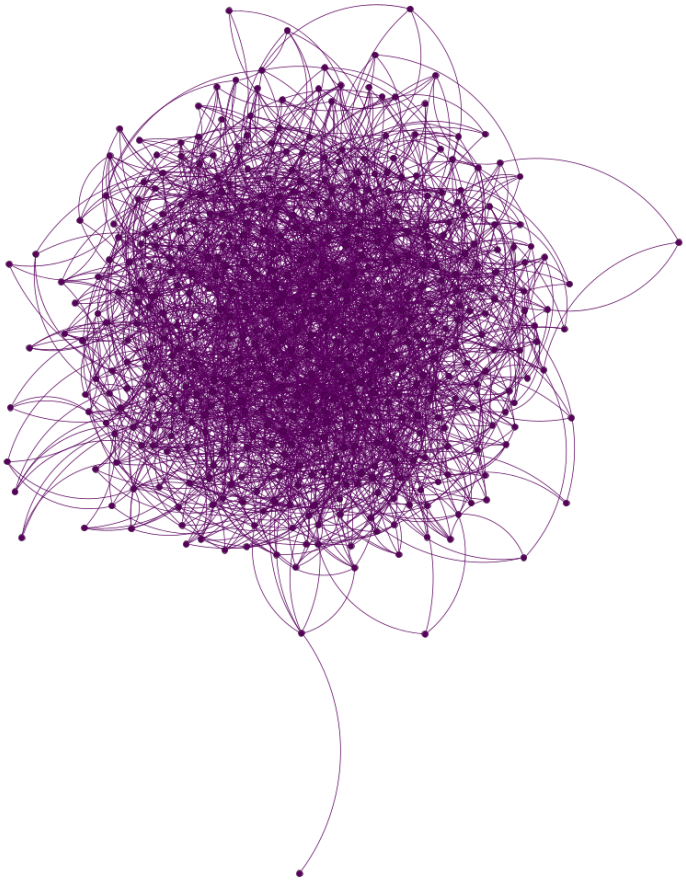
Figura 1: Grafos generados con el modelo Erdős–Rényi.



(a) $n = 50$

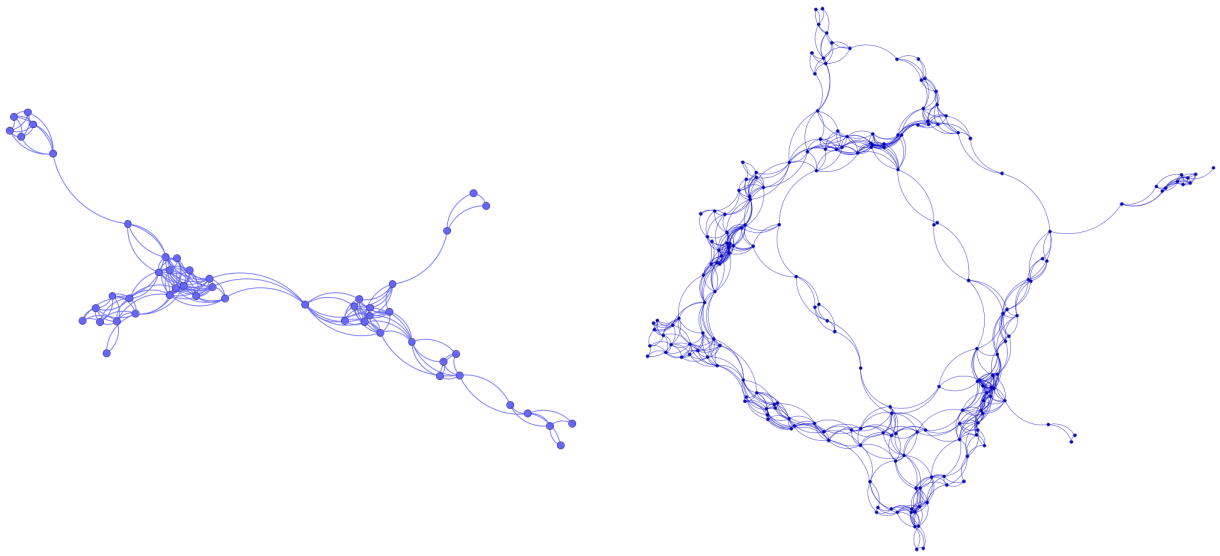


(b) $n = 200$

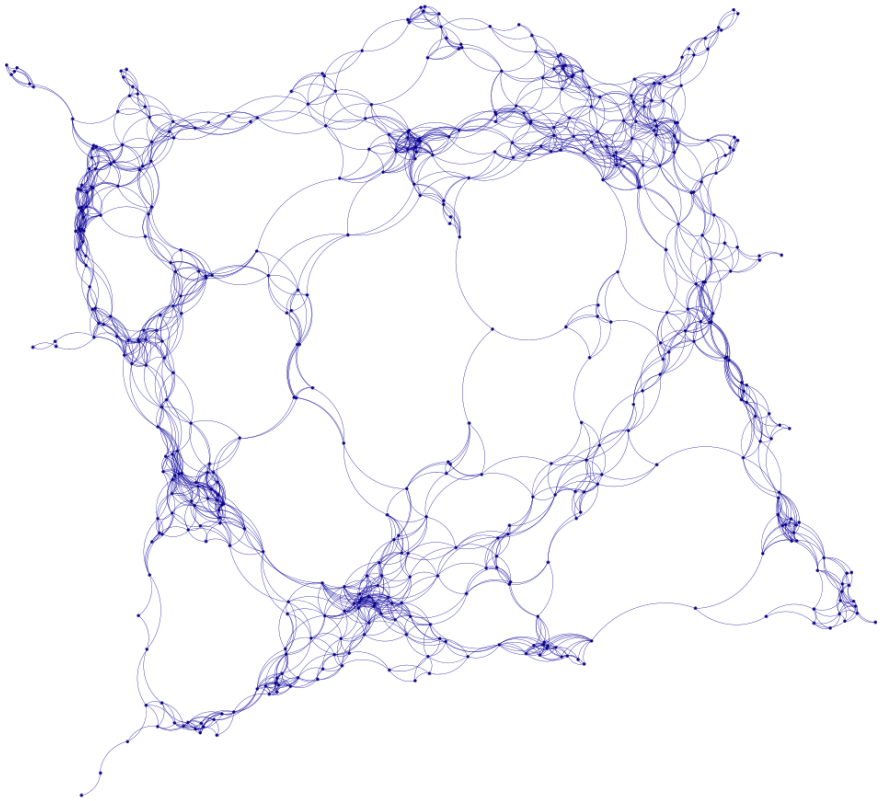


(c) $n = 500$

Figura 2: Grafos generados con el modelo Gilbert.

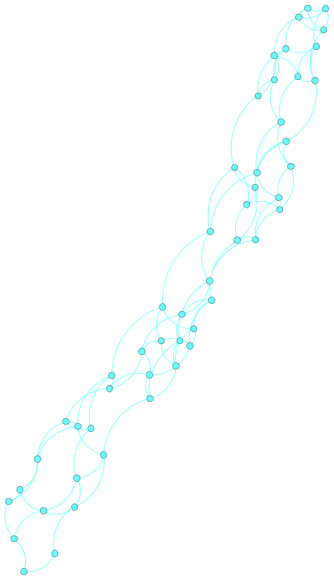


(a) $n = 50$ (b) $n = 200$

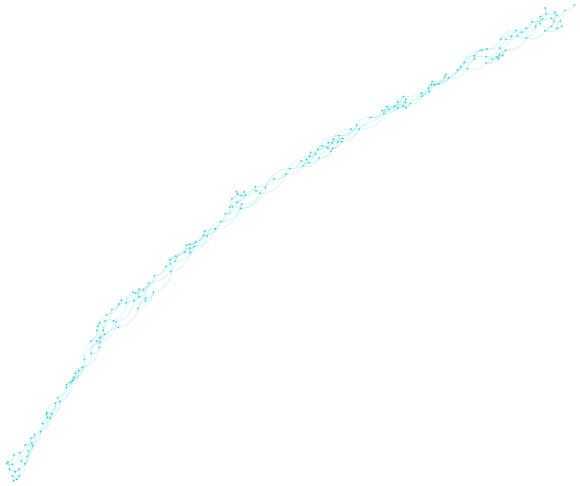


(c) $n = 500$

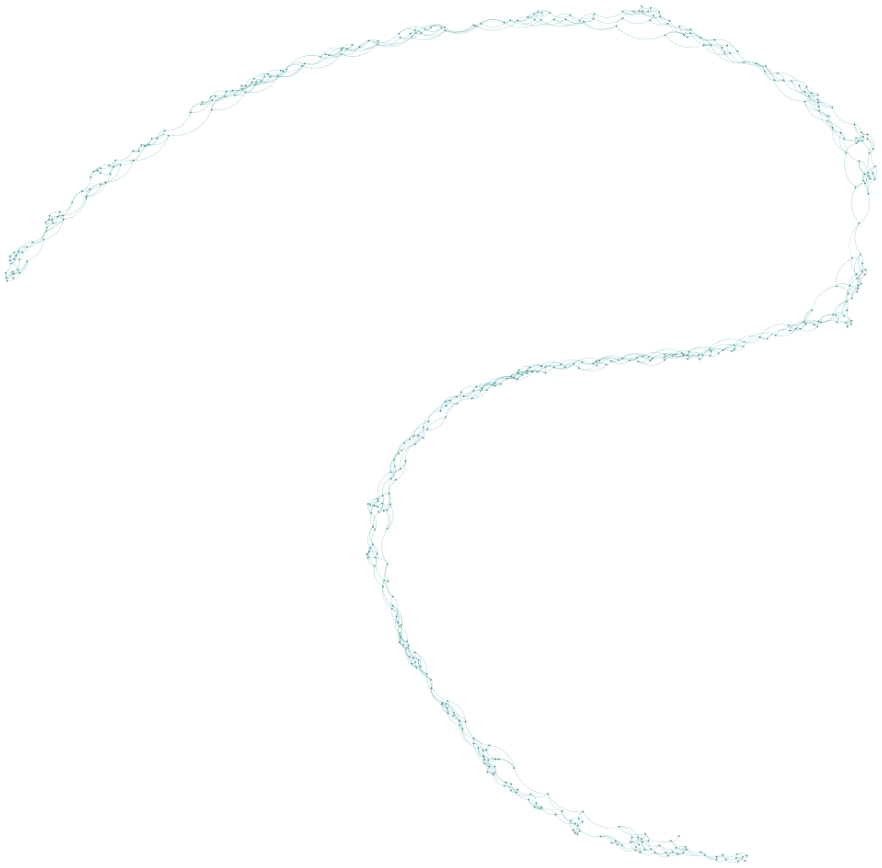
Figura 3: Grafos generados con el modelo geográfico simple.



(a) $n = 50$

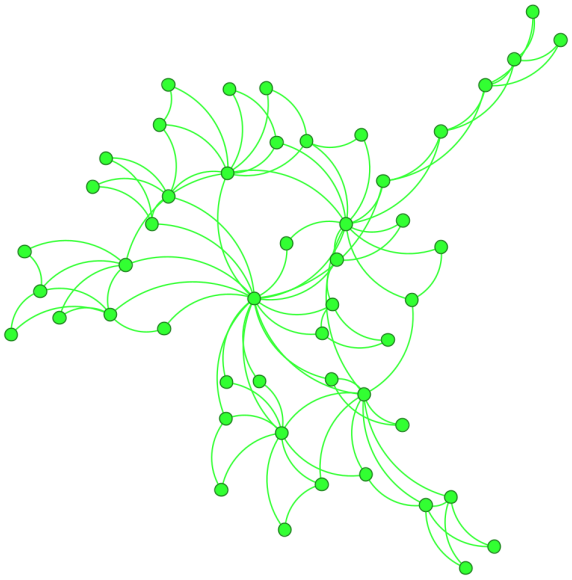


(b) $n = 200$

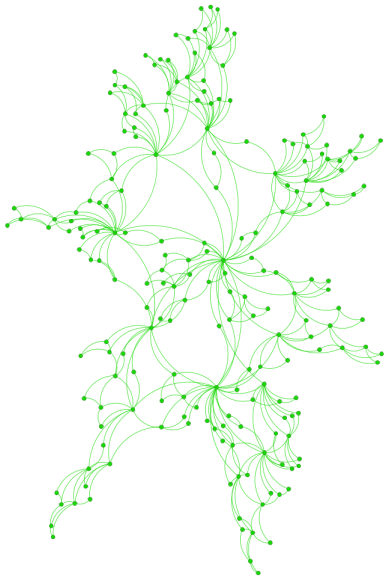


(c) $n = 500$

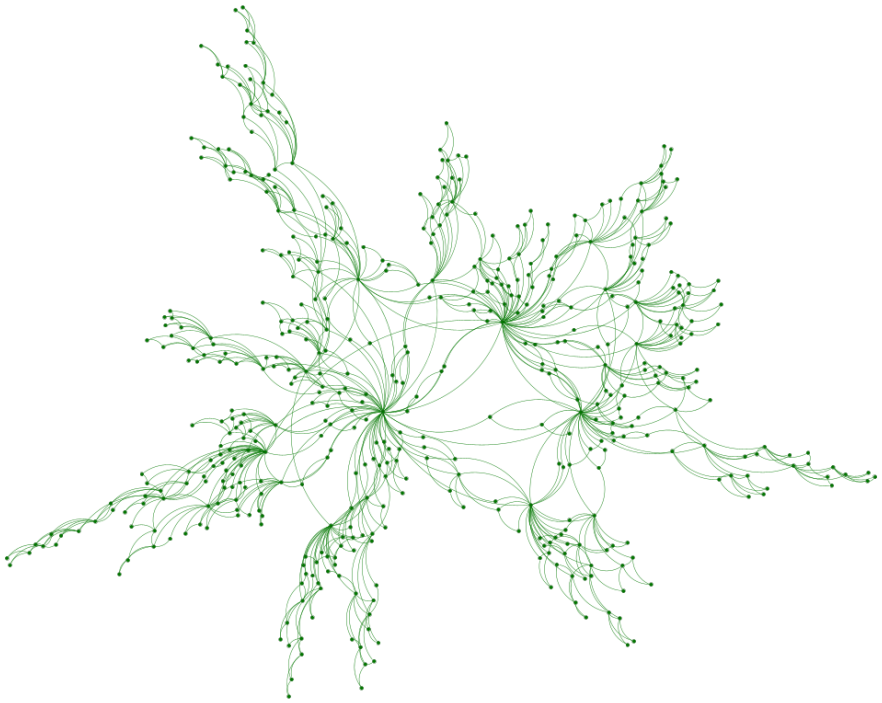
Figura 4: Grafos generados con el modelo Barabási–Albert.



(a) $n = 50$

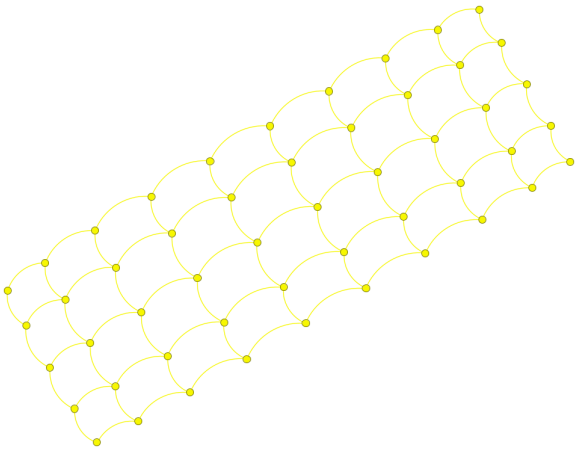


(b) $n = 200$

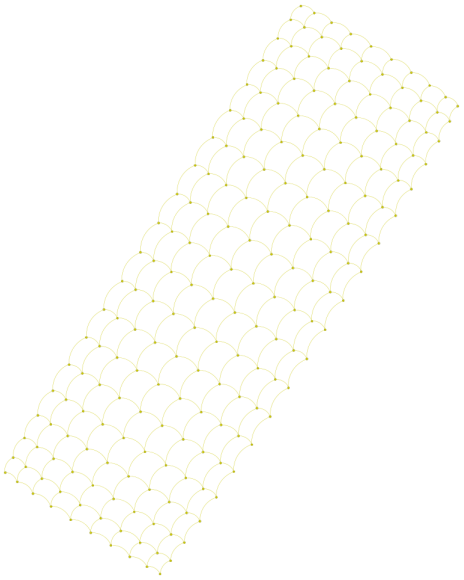


(c) $n = 500$

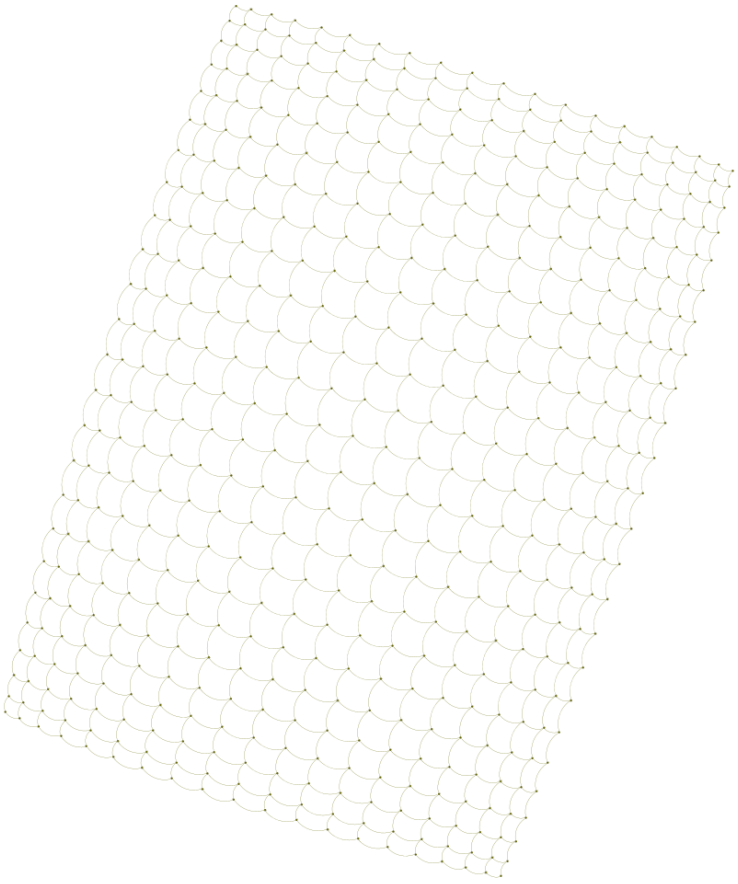
Figura 5: Grafos generados con el modelo Dorogovtsev–Mendes.



(a) $n = 50$



(b) $n = 200$



(c) $n = 500$

Figura 6: Grafos generados con el modelo malla.